Submitted By: RISCV_ASSEMBLERS

Name: V. V. S. Aneesh
Roll Number: 2025122002

Name: M. Sai Poojith
Roll Number: 2025122010

Name: Anumay Rai
Roll Number: 2025122013

Submitted On:

Date: 22nd February 2026

The International Institute of Information Technology, Hyderabad

Contents

1 Introduction **3**

2 Module Descriptions **4**

2.1 Program Counter (PC) 4
2.2 Register File 6
2.3 Instruction Memory 9
2.4 Control Unit 11
2.5 Immediate Generator 15
2.6 ALU Control 18
2.7 64-bit Arithmetic Logic Unit (ALU) 21
2.8 Data Memory 24
2.9 2:1 Multiplexer 27
2.10 64-bit Adder 28
2.11 Shift Left by 1 (SL1) 29

3 Sequential Processor (Top-Level Design) **30**

4 Testing and Results **34**

5 Conclusion **35**

Abstract

This project presents the design and implementation of a sequential (non-pipelined) RISC-V processor based on the RV64I instruction set. The processor is implemented in Verilog and simulated using iVerilog. It executes one instruction at a time in a single instruction cycle without pipelining or hazard handling.

The design includes key components such as the Program Counter, Instruction Memory, Register File, Control Unit, Immediate Generator, ALU Control, Arithmetic Logic Unit, and Data Memory. The processor supports arithmetic, logical, memory, and branch instructions including add, sub, addi, and, or, ld, sd, and beq. Each module is designed independently and integrated to form the complete datapath, and simulation results confirm correct functionality.

1 Introduction

RISC-V is an open-standard instruction set architecture(ISA) designed to provide a simple and modular framework for processor development. Sequential processors execute instructions one at a time, completing all stages of execution before moving to the next instruction.

The processor is designed using a modular approach, where each major component of the datapath is implemented as an independent Verilog module. The main components include the Program Counter (PC), Instruction Memory, Register File, Control Unit, Immediate Generator, ALU Control, Arithmetic Logic Unit (ALU), Data Memory, multiplexers, and adder blocks.

The Program Counter stores the address of the current instruction and updates sequentially or based on branch decisions. Instruction Memory provides the instruction corresponding to the PC address. The Control Unit decodes the instruction opcode and generates control signals for the datapath. The Register File supplies operands and stores computation results. The Immediate Generator extracts and sign-extends immediate values from instructions. The ALU performs arithmetic and logical operations, while Data Memory handles load and store operations. Multiplexers and adders are used to select inputs and compute addresses as required.

All modules operate synchronously with the clock signal, enabling the processor to execute the supported instructions correctly.

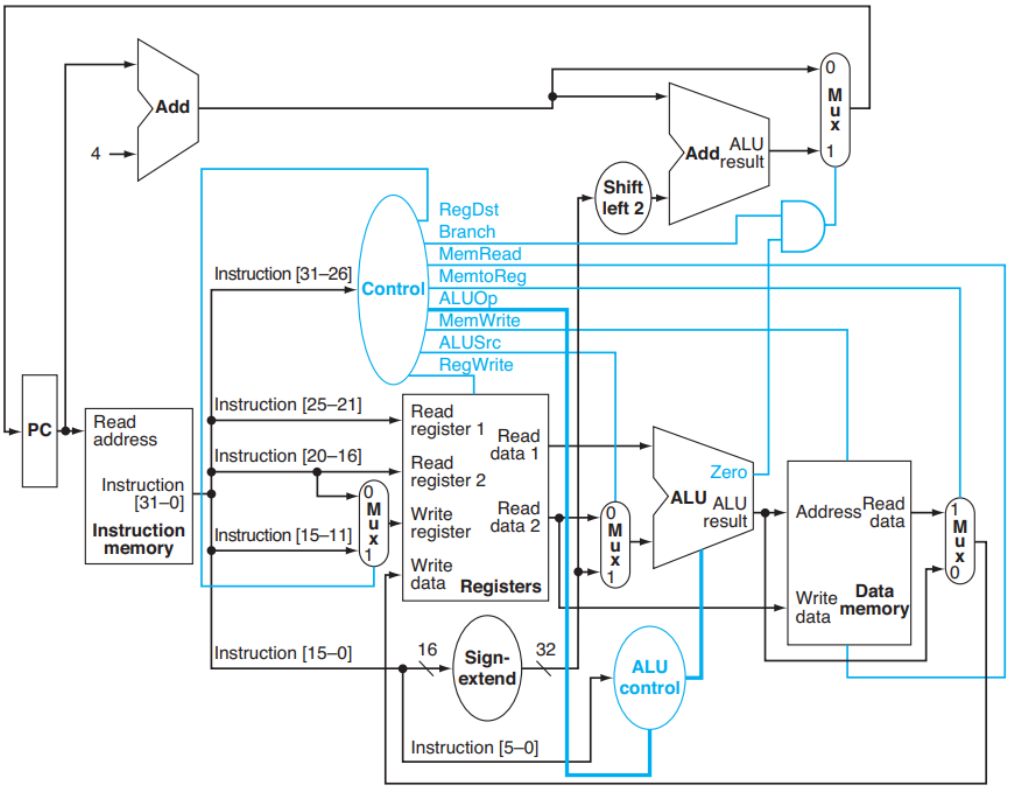

Figure 1: Complete Datapath of the Sequential processor

2 Module Descriptions

2.1 Program Counter (PC)

The Program Counter (PC) is a 64-bit register that stores the address of the current instruction being executed. It updates its value on every clock cycle and provides the address to the instruction memory.

When the reset signal is asserted, the PC is cleared to zero so that execution begins from the initial address. Otherwise, the PC loads the next instruction address from the input. Since the PC updates every clock cycle, a separate write enable signal is not required.

Implementation

Listing 1: Verilog Implementation of Program Counter

```
module pc(input clk, reset, input [63:0] pc_in, output [63:0] pc_out
   );
   reg [63:0] pc_reg;
   always @(posedge clk or posedge reset) begin
      if (reset) begin
         pc_reg <= 64'b0;
      end else begin
         pc_reg <= pc_in;
      end
   end
   assign pc_out = pc_reg;
endmodule
```

Testbench

Listing 2: Testbench for Program Counter

```
`timescale 1ns/1ps
`include "pc.v"

module pc_tb;
   reg clk;
   reg reset;
   reg [63:0] pc_in;
   wire [63:0] pc_out;

   initial begin
      clk = 0;
      forever #5 clk = ~clk;
   end

   pc uut(
      .clk(clk),
      .reset(reset),
```

```
        .pc_in(pc_in),
        .pc_out(pc_out)
    );

    task run_test;
        input [7:0] test_number;
        input [63:0] test_pc_in;
        input test_reset;
        input [63:0] expected_pc_out;
        begin
            pc_in = test_pc_in;
            reset = test_reset;

            @(posedge clk);
            #2;

            $display("Test %0d: pc_in = %h, reset = %b, pc_out = %h",
                    test_number, pc_in, reset, pc_out);
            if (pc_out === expected_pc_out)
                $display("PASS\n");
            else begin
                $display("FAIL");
                $display("Expected pc_out = %h\n", expected_pc_out);
            end
        end
    endtask

    initial begin
        $dumpfile("pc_tb.vcd");
        $dumpvars(0, pc_tb);
        run_test(1, 64'h00ADB12FDEADBEEF, 1, 64'h0);
        run_test(2, 64'h000000000ABCDEF4, 0,
                 64'h000000000ABCDEF4);

        #20 $finish;
    end
endmodule
```

Verification

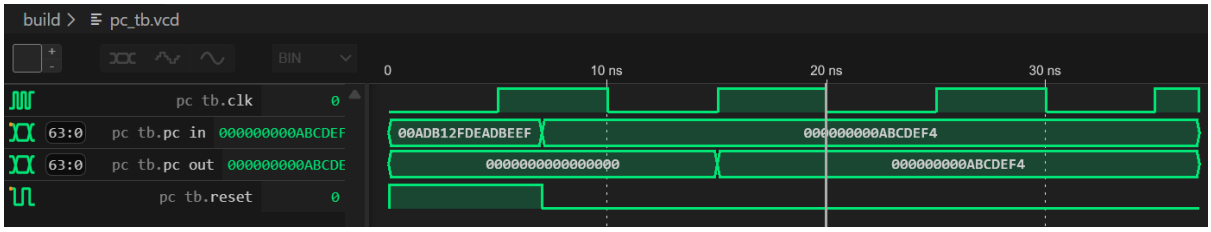

Figure 2: Simulation waveform of the Program Counter (PC) module

2.2 Register File

The Register File stores the general-purpose registers used by the processor for operand storage and result write-back. It consists of 32 registers, each 64 bits wide, following the RV64I architecture. The module provides two read ports and one write port, allowing simultaneous reading of two registers and writing to one register.

Read operations are combinational, meaning the output data is available immediately when the read register addresses are provided. Write operations occur on the rising edge of the clock when the write enable signal is asserted. When the reset signal is active, all registers are cleared to zero. Register zero is treated as a constant zero register and is not written to even if selected.

Implementation

Listing 3: Verilog Implementation of Register File

```
module register_file(input clk,reset,reg_write_en,
input[4:0] read_reg1, read_reg2, write_reg,
input[63:0] write_data,
output[63:0] read_data1, read_data2);

   reg[63:0] registers[31:0];

   integer i;
   initial begin
      for (i = 0; i<32; i=i+1) begin
         registers[i] = 64'b0;
      end
   end

   assign read_data1 = registers[read_reg1];
   assign read_data2 = registers[read_reg2];

   always @(posedge clk or posedge reset) begin
      if (reset) begin
         for (i=0; i<32; i=i+1) begin
            registers[i]<=64'b0;
         end
      end
      else if (reg_write_en && write_reg != 5'b0) begin
         registers[write_reg]<=write_data;
      end
   end
endmodule
```

Testbench

Listing 4: Testbench for Register File

```
`timescale 1ns/1ps
```

```
`include "register_file.v"

module register_file_tb;
   reg clk;
   reg reset;
   reg reg_write_en;
   reg [4:0] read_reg1, read_reg2, write_reg;
   reg [63:0] write_data;
   wire [63:0] read_data1, read_data2;

   register_file uut(
      .clk(clk),
      .reset(reset),
      .reg_write_en(reg_write_en),
      .read_reg1(read_reg1),
      .read_reg2(read_reg2),
      .write_reg(write_reg),
      .write_data(write_data),
      .read_data1(read_data1),
      .read_data2(read_data2));

   initial begin
      clk = 0;
      forever #5 clk = ~clk;
   end

   task run_test;
      input [4:0] tno;
      input t_reset;
      input t_reg_write_en;
      input [4:0] t_read_reg1;
      input [4:0] t_read_reg2;
      input [4:0] t_write_reg;
      input [63:0] t_write_data;
      input [63:0] expected1;
      input [63:0] expected2;
      begin
         @(negedge clk);
         reset = t_reset;
         reg_write_en = t_reg_write_en;
         read_reg1 = t_read_reg1;
         read_reg2 = t_read_reg2;
         write_reg = t_write_reg;
         write_data = t_write_data;
         @(posedge clk);
         #2;
         $display("Test %0d", tno);
         $display("Reset=%b WE=%b WR=%d WD=%h",
               reset, reg_write_en, write_reg, write_data);
         $display("RR1=%d RD1=%h", read_reg1, read_data1);
         $display("RR2=%d RD2=%h", read_reg2, read_data2);
```

```
        if (read_data1 === expected1 &&
           read_data2 === expected2)
           $display("PASS\n");
        else begin
           $display("FAIL");
           $display("Expected RD1=%h RD2=%h",
                 expected1, expected2);
           $display("Output RD1=%h RD2=%h\n",
                 read_data1, read_data2);
        end
     end
  endtask
  initial begin
     $dumpfile("register_file_tb.vcd");
     $dumpvars(0, register_file_tb);
     run_test(1, 0, 1, 0, 0, 0, 64'hFFFFFFFFFFFFFFFF,
            64'h0, 64'h0);
     run_test(2, 0, 1, 1, 0, 1, 64'hDEADBEEFDEADBEEF,
            64'hDEADBEEFDEADBEEF, 64'h0);
     run_test(3, 0, 1, 1, 2, 2, 64'hCAFEBABECAFEBABE,
            64'hDEADBEEFDEADBEEF,
            64'hCAFEBABECAFEBABE);
     run_test(4, 0, 0, 1, 2, 1, 64'h1111111111111111,
            64'hDEADBEEFDEADBEEF,
            64'hCAFEBABECAFEBABE);
     run_test(5, 0, 1, 0, 1, 0, 64'hFFFFFFFFFFFFFFFF,
            64'h0,
            64'hDEADBEEFDEADBEEF);
     run_test(6, 0, 1, 31, 0, 31, 64'h1234567890ABCDEF,
            64'h1234567890ABCDEF, 64'h0);
     #20 $finish;
  end
endmodule
```

Verification

Figure 3: Simulation waveform of the Register File module

2.3 Instruction Memory

The Instruction Memory stores the program instructions that are executed by the processor. It is a read-only memory module that outputs a 32-bit instruction corresponding to the address provided by the Program Counter. The memory is byte-addressed and organized as an array of 4096 bytes.

At initialization, the memory contents are loaded from an external text file containing hexadecimal instruction bytes. Each instruction occupies four consecutive bytes, and the module concatenates these bytes to form a 32-bit instruction word. The address input selects the starting byte location, and the next three bytes are combined to produce the complete instruction. This design follows a Big-Endian format, where the most significant byte is stored at the lowest address.

Implementation

Listing 5: Verilog Implementation of Instruction Memory

```
module Instruction_Memory(input[63:0] addr, output[31:0] instr);

   reg [7:0] memory[4095:0];
   integer i;

   initial begin
      for (i = 0; i < 4096; i = i + 1)
         memory[i] = 8'h00;

      $readmemh("../instructions.txt", memory);
   end

   assign instr = {memory[addr[11:0]],
                   memory[addr[11:0]+1],
                   memory[addr[11:0]+2],
                   memory[addr[11:0]+3]};
endmodule
```

Testbench

Listing 6: Testbench for Instruction Memory

```
`timescale 1ns/1ps
`include "Instruction_Memory.v"

module Instruction_Memory_tb;

   reg [63:0] addr;
   wire [31:0] instr;

   Instruction_Memory uut(
      .addr(addr),
      .instr(instr)
```

```
    );

    task run_test;
        input [4:0] tno;
        input [63:0] t_addr;
        input [31:0] expected;
        begin
            addr = t_addr;
            #6;

            $display("Test %0d", tno);
            $display("Address = %0d", addr);
            $display("Instruction = %h", instr);

            if (instr === expected)
                $display("PASS\n");
            else begin
                $display("FAIL");
                $display("Expected = %h\n", expected);
            end
        end
    endtask

    initial begin
        $dumpfile("Instruction_Memory_tb.vcd");
        $dumpvars(0, Instruction_Memory_tb);

        run_test(1, 64'd0, 32'h00500113);
        run_test(2, 64'd4, 32'h00A00193);
        run_test(3, 64'd8, 32'h003100B3);
        run_test(4, 64'd12, 32'h40310133);

        #20 $finish;
    end
endmodule
```

Verification

Figure 4: Simulation waveform of the Instruction Memory module

2.4 Control Unit

The Control Unit generates the control signals required for the operation of the datapath components. It decodes the 7-bit opcode field of the instruction and produces signals that determine data flow, ALU operation type, memory access, and register write behavior.

This module supports different instruction formats of the RV64I instruction set, including R-type, I-type, load, store, and branch instructions. For each instruction type, appropriate control signals are asserted to enable correct execution. The design uses a combinational logic block so that the control outputs change immediately with changes in the opcode.

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Figure 5: Control signal generation table for different instruction types in the RV64I sequential processor

Implementation

Listing 7: Verilog Implementation of Control Unit

```
module control(input[6:0] opcode,
output Branch, MemRead, MemToReg,
output[1:0] ALUOp,
output MemWrite, ALUSrc, reg_write_en);

   localparam R_type = 7'b0110011;
   localparam I_type = 7'b0010011;
   localparam S_type = 7'b0100011;
   localparam load_type = 7'b0000011;
   localparam B_type = 7'b1100011;

   reg Branch_reg, MemRead_reg, MemToReg_reg;
   reg MemWrite_reg, ALUSrc_reg, reg_write_en_reg;
   reg [1:0] ALUOp_reg;

   always @(*) begin
      case(opcode)

         R_type: begin
            Branch_reg = 0;
            MemRead_reg = 0;
            MemToReg_reg = 0;
            ALUOp_reg = 2'b10;
            MemWrite_reg = 0;
            ALUSrc_reg = 0;
```

```
            reg_write_en_reg = 1;
        end

        I_type: begin
            Branch_reg = 0;
            MemRead_reg = 0;
            MemToReg_reg = 0;
            ALUOp_reg = 2'b00;
            MemWrite_reg = 0;
            ALUSrc_reg = 1;
            reg_write_en_reg = 1;
        end

        load_type: begin
            Branch_reg = 0;
            MemRead_reg = 1;
            MemToReg_reg = 1;
            ALUOp_reg = 2'b00;
            MemWrite_reg = 0;
            ALUSrc_reg = 1;
            reg_write_en_reg = 1;
        end

        S_type: begin
            Branch_reg = 0;
            MemRead_reg = 0;
            MemToReg_reg = 0;
            ALUOp_reg = 2'b00;
            MemWrite_reg = 1;
            ALUSrc_reg = 1;
            reg_write_en_reg = 0;
        end

        B_type: begin
            Branch_reg = 1;
            MemRead_reg = 0;
            MemToReg_reg = 0;
            ALUOp_reg = 2'b01;
            MemWrite_reg = 0;
            ALUSrc_reg = 0;
            reg_write_en_reg = 0;
        end

        default: begin
            Branch_reg = 0;
            MemRead_reg = 0;
            MemToReg_reg = 0;
            ALUOp_reg = 2'b00;
            MemWrite_reg = 0;
            ALUSrc_reg = 0;
            reg_write_en_reg = 0;
```

```
        end
      endcase
   end

   assign Branch = Branch_reg;
   assign MemRead = MemRead_reg;
   assign MemToReg = MemToReg_reg;
   assign ALUOp = ALUOp_reg;
   assign MemWrite = MemWrite_reg;
   assign ALUSrc = ALUSrc_reg;
   assign reg_write_en = reg_write_en_reg;

endmodule
```

Testbench

Listing 8: Testbench for Control Unit

```
`timescale 1ns/1ps
`include "control.v"

module control_tb;

   reg [6:0] opcode;
   wire Branch;
   wire MemRead;
   wire MemToReg;
   wire [1:0] ALUOp;
   wire MemWrite;
   wire ALUSrc;
   wire reg_write_en;

   control uut(
      .opcode(opcode),
      .Branch(Branch),
      .MemRead(MemRead),
      .MemToReg(MemToReg),
      .ALUOp(ALUOp),
      .MemWrite(MemWrite),
      .ALUSrc(ALUSrc),
      .reg_write_en(reg_write_en)
   );

   task run_test;
      input [4:0] tno;
      input [6:0] t_opcode;
      input expBranch, expMemRead, expMemToReg;
      input [1:0] expALUOp;
      input expMemWrite, expALUSrc, expRegWrite;
      begin
```

```
            opcode = t_opcode;
            #5;

            $display("Test %0d", tno);
            $display("Opcode = %b", opcode);

            if (Branch===expBranch &&
               MemRead===expMemRead &&
               MemToReg===expMemToReg &&
               ALUOp===expALUOp &&
               MemWrite===expMemWrite &&
               ALUSrc===expALUSrc &&
               reg_write_en===expRegWrite)
               $display("PASS\n");
            else
               $display("FAIL\n");
         end
      endtask

      initial begin
         $dumpfile("control_tb.vcd");
         $dumpvars(0, control_tb);

         run_test(1, 7'b0110011, 0,0,0,2'b10,0,0,1);
         run_test(2, 7'b0010011, 0,0,0,2'b00,0,1,1);
         run_test(3, 7'b0000011, 0,1,1,2'b00,0,1,1);
         run_test(4, 7'b0100011, 0,0,0,2'b00,1,1,0);
         run_test(5, 7'b1100011, 1,0,0,2'b01,0,0,0);

         #20 $finish;
      end
   endmodule
```

Verification

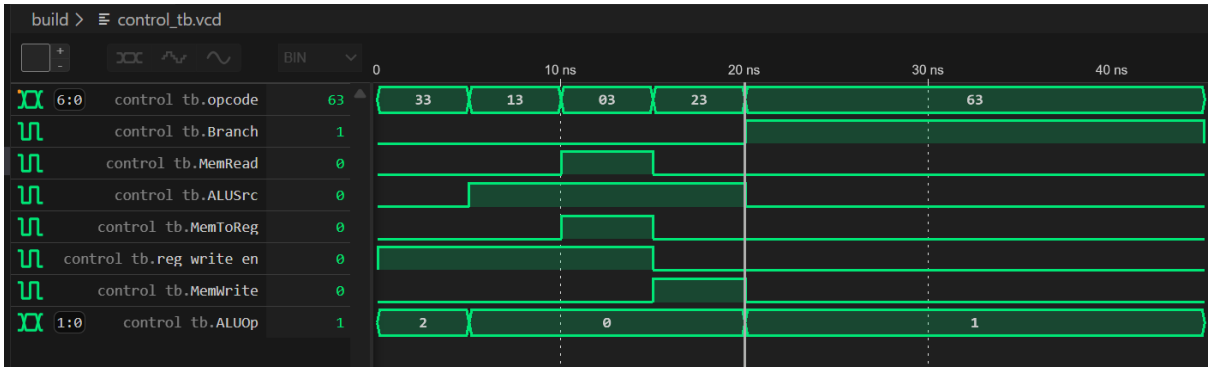

Figure 6: Simulation waveform of the Control Unit module

2.5 Immediate Generator

The Immediate Generator extracts the immediate value from a 32-bit instruction and converts it into a 64-bit signed value. Immediate values are used by various instruction types for arithmetic operations, memory addressing, and branch target calculation.

Different instruction formats in the RV64I architecture encode immediates in different bit positions. This module decodes the opcode to determine the instruction format and extracts the corresponding immediate field. The extracted value is sign-extended to 64 bits so that it can be used correctly in arithmetic operations.

The module supports I-type, load-type, S-type, and B-type instructions. Combinational logic is used to ensure that the output updates immediately when the input instruction changes.

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

Figure 7: Instruction encoding formats in RV64I architecture highlighting immediate field positions for different instruction types.

Implementation

Listing 9: Verilog Implementation of Immediate Generator

```
module Immediate_Generation(input[31:0] instr, output[63:0] imm);

   wire [6:0] opcode = instr[6:0];
   reg [63:0] imm_reg;

   localparam I_type = 7'b0010011;
   localparam S_type = 7'b0100011;
   localparam load_type = 7'b0000011;
   localparam B_type = 7'b1100011;

   always @(*) begin
      case (opcode)

         I_type, load_type:
            imm_reg = {{52{instr[31]}}, instr[31:20]};

         S_type:
            imm_reg = {{52{instr[31]}},
                    instr[31:25], instr[11:7]};

```

```
        B_type:
            imm_reg = {{52{instr[31]}},
                    instr[31], instr[7],
                    instr[30:25], instr[11:8]};

        default:
            imm_reg = 64'b0;
    endcase
  end

  assign imm = imm_reg;
endmodule
```

Testbench

Listing 10: Testbench for Immediate Generator

```
`timescale 1ns/1ps
`include "Immediate_Generation.v"

module Immediate_Generation_tb;

  reg [31:0] instr;
  wire [63:0] imm;

  Immediate_Generation uut(
    .instr(instr),
    .imm(imm)
  );

  task run_test;
    input [4:0] tno;
    input [31:0] t_instr;
    input [63:0] expected;
    begin
      instr = t_instr;
      #5;

      $display("Test %0d", tno);
      $display("Instruction = %h", instr);
      $display("Immediate = %h", imm);

      if (imm === expected)
        $display("PASS\n");
      else begin
        $display("FAIL");
        $display("Expected = %h\n", expected);
      end
    end
  endtask
```

```

    initial begin
        $dumpfile("Immediate_Generation_tb.vcd");
        $dumpvars(0, Immediate_Generation_tb);

        // I-type
        run_test(1,
            32'b000000000010_00010_000_00001_0010011,
            64'd2);

        // load-type
        run_test(2,
            32'b111111111111_00010_000_00001_0000011,
            64'hFFFFFFFFFFFFFFFF);

        // S-type
        run_test(3,
            32'b0000000_00010_00001_010_01000_0100011,
            64'd8);

        // B-type
        run_test(4,
            32'b0000000_00010_00001_000_00100_1100011,
            64'd2);

        #20 $finish;
    end
endmodule
```

Verification

Figure 8: Simulation waveform of the Immediate Generator module

2.6 ALU Control

The ALU Control unit generates the control signal required by the Arithmetic Logic Unit (ALU) to perform the correct operation. It receives the ALUOp signal from the main Control Unit and instruction function bits as inputs.

The ALUOp signal indicates the instruction category, while the function fields determine the exact operation for R-type instructions. Based on these inputs, the module produces a 4-bit control signal that selects the operation performed by the ALU.

Load and store instructions require addition for effective address calculation, branch instructions require subtraction for comparison, and R-type instructions select operations such as ADD, SUB, AND, and OR depending on the funct3 and funct7 bits. The module uses combinational logic so that the control signal updates immediately when inputs change.

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
ld	00	load doubleword	XXXXXXX	XXX	add	0010
sd	00	store doubleword	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

Figure 9: ALU Control signal generation table showing ALUOp, funct3, and funct7 decoding for different instruction types

Implementation

Listing 11: Verilog Implementation of ALU Control

```
module alu_control(input[1:0] ALUOp, input[3:0] Ins , output[3:0]
   ALUControl);

   reg[3:0] ALUControl_reg;
   wire[2:0] funct3;
   wire funct7_5;

   assign funct3 = Ins[2:0];
   assign funct7_5 = Ins[3];

always @(*) begin
   case (ALUOp)

      2'b00: ALUControl_reg = 4'b0010; // ADD for Ld and Sd
      2'b01: ALUControl_reg = 4'b0110; // SUB for BEQ

      2'b10: begin // R-type instructions
         case (funct3)
```

```

            3'b000: begin
               if (funct7_5 == 1'b1)
                  ALUControl_reg = 4'b0110; // SUB
               else
                  ALUControl_reg = 4'b0010; // ADD
            end

            3'b111: ALUControl_reg = 4'b0000; // AND
            3'b110: ALUControl_reg = 4'b0001; // OR

            default: ALUControl_reg = 4'b0010;
         endcase
      end

      default: ALUControl_reg = 4'b0010;
   endcase
end

assign ALUControl = ALUControl_reg;

endmodule
```

Testbench

Listing 12: Testbench for ALU Control

```
`timescale 1ns/1ps
`include "alu_control.v"

module alu_control_tb;

   reg [1:0] ALUOp;
   reg [3:0] Ins;
   wire [3:0] ALUControl;

   alu_control uut(
      .ALUOp(ALUOp),
      .Ins(Ins),
      .ALUControl(ALUControl)
   );

   task run_test;
      input [4:0] tno;
      input [1:0] t_ALUOp;
      input [3:0] t_Ins;
      input [3:0] expected;
      begin
         ALUOp = t_ALUOp;
         Ins = t_Ins;
```

```
        #5;

        $display("Test %0d", tno);
        $display("ALUOp=%b Ins=%b  ALUControl=%b", ALUOp, Ins,
           ALUControl);

        if (ALUControl === expected)
           $display("PASS\n");
        else begin
           $display("FAIL");
           $display("Expected = %b\n", expected);
        end
     end
  endtask

  initial begin
     $dumpfile("alu_control_tb.vcd");
     $dumpvars(0, alu_control_tb);

     run_test(1, 2'b00, 4'b0000, 4'b0010);
     run_test(2, 2'b01, 4'b0000, 4'b0110);
     run_test(3, 2'b10, 4'b0000, 4'b0010);
     run_test(4, 2'b10, 4'b1000, 4'b0110);
     run_test(5, 2'b10, 4'b0111, 4'b0000);
     run_test(6, 2'b10, 4'b0110, 4'b0001);

     #20 $finish;
  end
endmodule
```

Verification

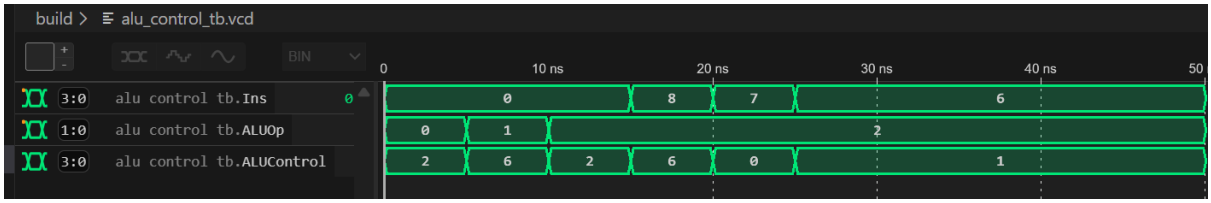

Figure 10: Simulation waveform of the ALU Control module showing ALUOp decoding and generation of ALU control signals

2.7 64-bit Arithmetic Logic Unit (ALU)

The 64-bit Arithmetic Logic Unit (ALU) performs arithmetic and logical operations on two 64-bit input operands. It is a fundamental component of the processor datapath and executes operations specified by the control signals generated from the ALU Control unit.

The module supports addition, subtraction, bitwise AND, and bitwise OR operations. Each operation is implemented using dedicated 64-bit submodules, and the required result is selected based on the opcode input. Along with the computed result, the ALU also generates a zero flag that indicates whether the output value is zero. This flag is primarily used for branch decisions.

Combinational logic is used to ensure that the output changes immediately in response to any change in inputs or opcode.

Implementation

Listing 13: Verilog Implementation of 64-bit ALU

```
module alu_64_bit(input [63:0]a,b, input [3:0]opcode, output [63:0]
   result, output zero_flag);

   localparam ADD_Oper = 4'b0010,
            OR_Oper = 4'b0001,
            AND_Oper = 4'b0000,
            SUB_Oper = 4'b0110;

   wire [63:0] add_out, sub_out, and_out, or_out;
   wire add_z, sub_z, and_z, or_z;

   add64 add_u(.a(a), .b(b), .sum(add_out), .zero(add_z));
   sub64 sub_u(.a(a), .b(b), .sum(sub_out), .zero(sub_z));
   and64 and_u(.a(a), .b(b), .out(and_out), .zero(and_z));
   or64 or_u (.a(a), .b(b), .out(or_out), .zero(or_z));

   reg [63:0] res_sel;
   reg z_sel;

   always @(*) begin
      case(opcode)

         ADD_Oper: begin
            res_sel = add_out;
            z_sel = add_z;
         end

         SUB_Oper: begin
            res_sel = sub_out;
            z_sel = sub_z;
         end

         AND_Oper: begin
            res_sel = and_out;
```

```
            z_sel = and_z;
         end

         OR_Oper: begin
            res_sel = or_out;
            z_sel = or_z;
         end

         default: begin
            res_sel = 64'b0;
            z_sel = 1'b1;
         end

      endcase
   end

   assign result = res_sel;
   assign zero_flag = z_sel;

endmodule
```

Testbench

Listing 14: Testbench for 64-bit ALU

```
`timescale 1ns/1ps
`include "alu.v"

module alu_64_bit_tb;
   reg [63:0] a, b;
   reg [3:0] opcode;
   wire [63:0] result;
   wire zero_flag;
   integer pass_count = 0, total_tests = 4;

   localparam ADD_Oper = 4'b0010,
            OR_Oper = 4'b0001,
            AND_Oper = 4'b0000,
            SUB_Oper = 4'b0110;

   alu_64_bit uut(
      .a(a),
      .b(b),
      .opcode(opcode),
      .result(result),
      .zero_flag(zero_flag)
   );

   task run_test;
      input [7:0] test_number;
```

```
        input [63:0] test_a, test_b, expected_result;
        input [3:0] test_opcode;
        input exp_zero;
        begin
            a = test_a;
            b = test_b;
            opcode = test_opcode;
            #10;

            if (result === expected_result && zero_flag === exp_zero)
                pass_count = pass_count + 1;
        end
    endtask

    initial begin
        $dumpfile("alu_tb.vcd");
        $dumpvars(0, alu_64_bit_tb); pass_count = 0;

        run_test(1, 64'h0000000000000000, 64'h0000000000000000, 64'h0000000000000000, ADD_Oper, 1);
        run_test(2, 64'h0000000000000000, 64'h0000000000000000, 64'h0000000000000000, SUB_Oper, 1);
        run_test(3, 64'hFFFFFFFFFFFFFFFF, 64'h0000000000000000, 64'h0000000000000000, AND_Oper, 1);
        run_test(4, 64'h0000000000000000, 64'h0000000000000000, 64'h0000000000000000, OR_Oper, 1);

        #10 $finish;
    end
endmodule
```

Verification

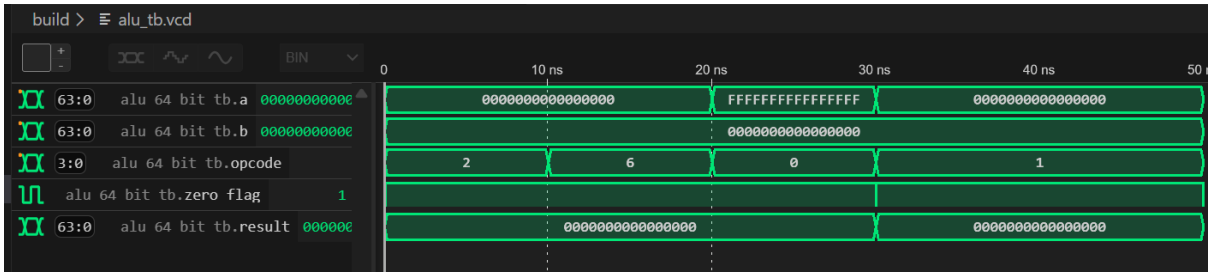

Figure 11: Simulation waveform of the Arithmetic Logic Unit (ALU) showing arithmetic and logical operations

2.8 Data Memory

The Data Memory module provides storage for data values used during load and store operations. It is a byte-addressable memory that allows reading and writing of 64-bit data words. The module receives control signals from the Control Unit to determine whether a memory operation should be performed.

The memory consists of 1024 bytes, organized as an array of 8-bit locations. For write operations, a 64-bit input value is split into eight bytes and stored across consecutive memory addresses in big-endian format. For read operations, eight consecutive bytes are combined to produce a 64-bit output value.

Write operations occur synchronously on the rising edge of the clock when the MemWrite signal is asserted. A reset signal initializes all memory locations to zero. Read operations are controlled by the MemRead signal and provide the requested data on the output bus.

Implementation

Listing 15: Verilog Implementation of Data Memory

```
module Data_Memory(input clk,reset,MemRead,MemWrite,
              input[9:0] address,
              input[63:0] write_data,
              output[63:0] read_data);

   reg[7:0] Dmemory[1023:0];
   integer k;

   initial begin
      for (k = 0; k < 1024; k = k + 1)
         Dmemory[k] = 8'h00;
   end

   always @(posedge clk) begin
      if (reset) begin
         for (k = 0; k < 1024; k = k + 1)
            Dmemory[k] <= 8'h0;
      end else begin
         if (MemWrite) begin
            Dmemory[address] <= write_data[63:56];
            Dmemory[address + 1] <= write_data[55:48];
            Dmemory[address + 2] <= write_data[47:40];
            Dmemory[address + 3] <= write_data[39:32];
            Dmemory[address + 4] <= write_data[31:24];
            Dmemory[address + 5] <= write_data[23:16];
            Dmemory[address + 6] <= write_data[15:8];
            Dmemory[address + 7] <= write_data[7:0];
         end
      end
   end

   assign read_data = MemRead ?
      {Dmemory[address], Dmemory[address+1],
```

```
        Dmemory[address+2], Dmemory[address+3],
        Dmemory[address+4], Dmemory[address+5],
        Dmemory[address+6], Dmemory[address+7]}
       : 64'b0;

endmodule
```

Testbench

Listing 16: Testbench for Data Memory

```
`timescale 1ns/1ps
`include "Data_Memory.v"

module Data_Memory_tb;

   reg clk;
   reg reset;
   reg MemRead;
   reg MemWrite;
   reg [9:0] address;
   reg [63:0] write_data;
   wire [63:0] read_data;

   Data_Memory uut(
      .clk(clk),
      .reset(reset),
      .MemRead(MemRead),
      .MemWrite(MemWrite),
      .address(address),
      .write_data(write_data),
      .read_data(read_data)
   );

   initial begin
      clk = 0;
      forever #5 clk = ~clk;
   end

   task run_test;
      input [4:0] tno;
      input t_reset;
      input t_MemWrite;
      input t_MemRead;
      input [9:0] t_address;
      input [63:0] t_write_data;
      input [63:0] expected;
      begin
         @(negedge clk);
         reset = t_reset;
```

```
        MemWrite = t_MemWrite;
        MemRead = t_MemRead;
        address = t_address;
        write_data = t_write_data;

        @(posedge clk);
        #2;

        if (read_data === expected)
           $display("Test %0d PASS", tno);
        else
           $display("Test %0d FAIL", tno);
     end
  endtask

  initial begin
     $dumpfile("Data_Memory_tb.vcd");
     $dumpvars(0, Data_Memory_tb);

     run_test(1, 1'b0, 1'b1, 1'b1,
            10'd8, 64'hDEADBEEFCAFEBABE,
            64'hDEADBEEFCAFEBABE);

     run_test(2, 1'b0, 1'b0, 1'b1,
            10'd8, 64'h1234567890ABCDEF,
            64'hDEADBEEFCAFEBABE);

     run_test(3, 1'b0, 1'b0, 1'b0,
            10'd16, 64'd0,
            64'd0);

     #20 $finish;
  end
endmodule
```

Verification

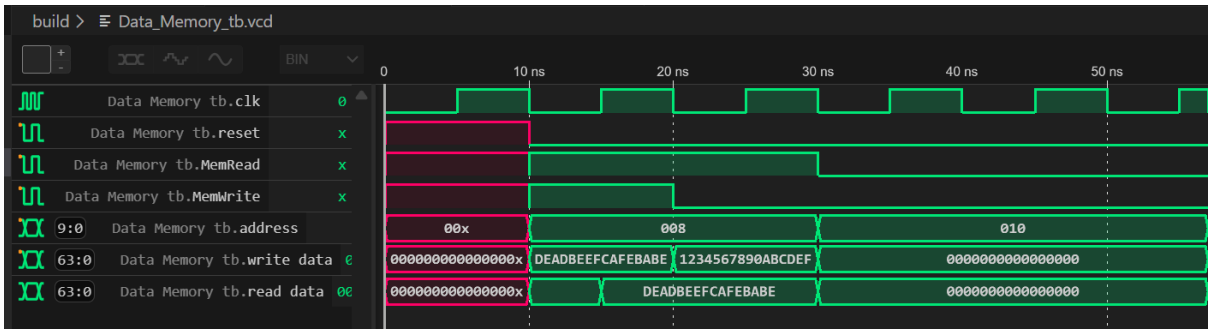

Figure 12: Simulation waveform of the Data Memory module showing load and store operations

2.9 2:1 Multiplexer

The 2:1 Multiplexer selects one of two 64-bit input signals based on a control signal. Multiplexers are widely used in the processor datapath to choose between alternative data sources, such as selecting between register values and immediate values for ALU inputs or choosing data for write-back.

This module receives two 64-bit inputs and a single selection signal. If the select signal is low, the first input is passed to the output; otherwise, the second input is selected. The module is implemented using combinational logic so that the output updates immediately when any input or the select signal changes.

Implementation

Listing 17: Verilog Implementation of 2:1 Multiplexer

```
module mux2_1(input[63:0] a,b, input sel, output[63:0] out);
   assign out = sel ? b : a;
endmodule
```

Testbench

Listing 18: Testbench for 2:1 Multiplexer

```
`timescale 1ns/1ps
`include "mux2_1.v"
module mux2_1_tb;
   reg [63:0] a, b;reg sel;wire [63:0] out;
   mux2_1 uut (.a(a), .b(b), .sel(sel), .out(out));
   initial begin
      $dumpfile("mux2_1_tb.vcd");
      $dumpvars(0, mux2_1_tb);
      a = 64'hAAAA; b = 64'hBBBB; sel = 0; #10;
      a = 64'h123456789ABCDEF0; b = 64'h0; sel = 1; #10;
      $finish;
   end
endmodule
```

Verification

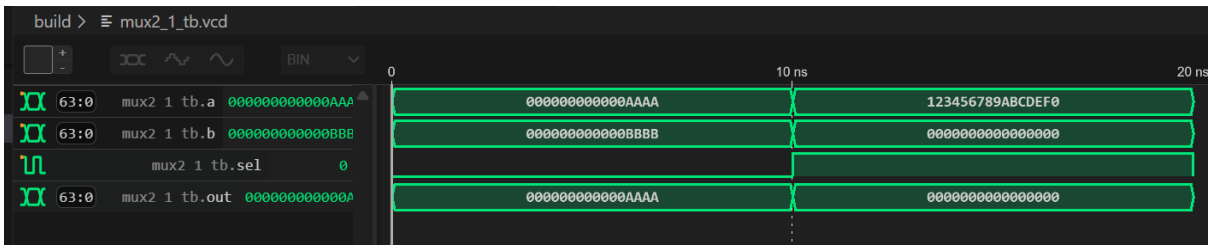

Figure 13: Simulation waveform of the 2-to-1 Multiplexer (MUX) module showing output selection based on the select signal

2.10 64-bit Adder

The 64-bit Adder performs arithmetic addition of two 64-bit input operands and produces a 64-bit result. Adders are fundamental components in processor datapaths and are used for operations such as Program Counter (PC) increment (PC + 4), effective address calculation, and branch target computation.

This module takes two 64-bit inputs and generates their sum using combinational logic. Since the design is purely combinational, the output updates immediately whenever the input values change.

Implementation

Listing 19: Verilog Implementation of 64-bit Adder

```
module adder64(input[63:0] a,b ,output [63:0] sum);
   assign sum = a + b;
endmodule
```

Testbench

Listing 20: Testbench for 64-bit Adder

```
`timescale 1ns/1ps
`include "adder64.v"
module adder64_tb;
   reg [63:0] a, b;wire [63:0] sum;
   adder64 uut (.a(a), .b(b), .sum(sum));
   initial begin
      $dumpfile("adder64_tb.vcd");
      $dumpvars(0, adder64_tb);
      a = 64'd10; b = 64'd20; #10;
      a = 64'hFFFFFFFFFFFFFFF0; b = 64'd16; #10;
      $finish;
   end
endmodule
```

Verification

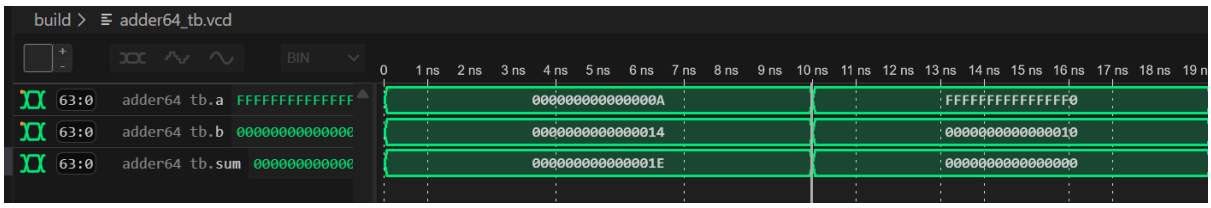

Figure 14: Simulation waveform of the 64-bit Adder module showing arithmetic addition of inputs a and b

2.11 Shift Left by 1 (SL1)

The Shift Left by 1 module performs a logical left shift of a 64-bit input value by one bit position. This operation multiplies the input value by two and appends a zero in the least significant bit position. Such a shift operation is commonly used in branch address computation, where the immediate offset must be aligned to instruction boundaries.

The module is implemented as a simple combinational circuit without any clock dependency. The output updates immediately whenever the input changes.

Implementation

Listing 21: Verilog Implementation of Shift Left by 1 Module

```
module sl1(input[63:0] in, output[63:0] out);
   assign out = {in[62:0], 1'b0};
endmodule
```

Testbench

Listing 22: Testbench for Shift Left by 1 (SL1) Module

```
`timescale 1ns/1ps
`include "sl1.v"

module sl1_tb;
   reg [63:0] in;wire [63:0] out;
   sl1 uut (.in(in), .out(out));
   initial begin
      $dumpfile("sl1_tb.vcd");
      $dumpvars(0, sl1_tb);
      in = 64'h0000000000000001; #10;
      in = 64'h000000000000000F; #10;
      in = 64'h8000000000000000; #10;
      $finish;
   end
endmodule
```

Verification

Figure 15: Simulation waveform of the 64-bit Adder module showing arithmetic addition of inputs `a` and `b`

3 Sequential Processor (Top-Level Design)

The Sequential Processor module represents the complete integration of all datapath components to form a working non-pipelined RISC-V processor. It executes instructions sequegntially, completing all stages of instruction execution — fetch, decode, execute, memory access, and write-back — within a single instruction cycle.

This top-level module connects the Program Counter, Instruction Memory, Control Unit, Register File, Immediate Generator, ALU Control, ALU, Data Memory, multiplexers, adders, and branch logic. The control signals generated by the Control Unit determine the flow of data through the datapath and enable correct execution of different instruction types.

Branch handling is implemented by computing the branch target address and selecting it based on the branch condition. Memory operations and register write-back are controlled using multiplexers. The processor operates synchronously with the clock signal and resets to the initial state when the reset signal is asserted.

Implementation

Listing 23: Top-Level Sequential Processor Implementation

```
module seq(input clk, reset);

   wire [63:0] pc_in, pc_out;
   wire reg_write_en;
   wire [63:0] write_data, read_data1, read_data2;
   wire [31:0] instr;
   wire Branch, MemRead, MemToReg, MemWrite, ALUSrc;
   wire [1:0] ALUOp;
   wire [3:0] ALUControl;
   wire [63:0] imm, imm_shifted;
   wire [63:0] alu_in2, alu_result;
   wire zero_flag;
   wire [63:0] read_data_mem;
   wire [63:0] mux_pc_1, mux_pc_2;
   wire Branch_and_zero;

   pc pc_inst(.clk(clk), .reset(reset),
            .pc_in(pc_in), .pc_out(pc_out));

   Instruction_Memory Instruction_Memory_inst(
      .addr(pc_out), .instr(instr));

   control control_inst(
      .opcode(instr[6:0]), .Branch(Branch),
      .MemRead(MemRead), .MemToReg(MemToReg),
      .ALUOp(ALUOp), .MemWrite(MemWrite),
      .ALUSrc(ALUSrc), .reg_write_en(reg_write_en));

   register_file register_file_inst(
      .clk(clk), .reset(reset),
```

```
        .reg_write_en(reg_write_en),
        .read_reg1(instr[19:15]),
        .read_reg2(instr[24:20]),
        .write_reg(instr[11:7]),
        .write_data(write_data),
        .read_data1(read_data1),
        .read_data2(read_data2));

    Immediate_Generation immediate_generation_inst(
        .instr(instr), .imm(imm));

    alu_control alu_control_inst(
        .ALUOp(ALUOp),
        .Ins({instr[30], instr[14:12]}),
        .ALUControl(ALUControl));

    Data_Memory Data_Memory_inst(
        .clk(clk), .reset(reset),
        .MemRead(MemRead), .MemWrite(MemWrite),
        .address(alu_result[9:0]),
        .write_data(read_data2),
        .read_data(read_data_mem));

    adder64 adderpc_inst(
        .a(pc_out), .b(64'd4), .sum(mux_pc_1));

    sl1 sl1_inst(.in(imm), .out(imm_shifted));

    adder64 adderbranch_inst(
        .a(pc_out), .b(imm_shifted), .sum(mux_pc_2));

    and2 and_branch_inst(
        .a(Branch), .b(zero_flag),
        .out(Branch_and_zero));

    mux2_1 mux_pc_inst(
        .a(mux_pc_1), .b(mux_pc_2),
        .sel(Branch_and_zero), .out(pc_in));

    mux2_1 mux_alu_inst(
        .a(read_data2), .b(imm),
        .sel(ALUSrc), .out(alu_in2));

    mux2_1 mux_write_data_inst(
        .a(alu_result), .b(read_data_mem),
        .sel(MemToReg), .out(write_data));

    alu_64_bit alu_inst(
        .a(read_data1), .b(alu_in2),
        .opcode(ALUControl),
        .result(alu_result),
```

```
      .zero_flag(zero_flag));

endmodule
```

Testbench

Listing 24: Testbench for Sequential Processor

```


`timescale 1ns/1ps
`include "seq.v"

module seq_tb;

   reg clk;
   reg reset;
   integer i;
   integer instr_count;
   integer outfile;

   seq uut (
      .clk(clk),
      .reset(reset)
   );

   initial begin
      instr_count = 0;
      clk = 0;
      forever #5 clk = ~clk;
   end

   initial begin
   reset = 1;
   #11;
   reset = 0;
   end

   always @(posedge clk) begin
      if (!reset)
         instr_count = instr_count + 1;
   end

   initial begin
      $dumpfile("seq_tb.vcd");
      $dumpvars(0, seq_tb);
      wait(uut.instr == 32'h00000000);
      @(posedge clk);
      #2

```

```
        outfile = $fopen("register_file.txt", "w");

        for (i = 0; i < 32; i = i + 1)
            $fdisplay(outfile, "%016h", uut.register_file_inst.
                registers[i]);

        $fdisplay(outfile, "%0d", instr_count);

        $fclose(outfile);
        $finish;

    end
endmodule
```

Verification

```
addi x2, x0, 5          # x2 = 5
addi x3, x0, 10         # x3 = 10
add x1, x2, x3          # x1 = x2 + x3 = 15
sub x2, x2, x3          # x2 = x2 - x3 = -5
and x4, x3, x3          # x4 = x3 & x3 = 10
and x5, x3, x4          # x5 = x3 & x4 = 10
or x6, x2, x4           # x6 = x2 | x4 = -5
or x7, x2, x3           # x7 = x2 | x3 = -5
sd x1, 0(x5)            # x1 is stored at memory address x5 + 0 (10 + 0 = 10)
ld x10, 0(x5)           # x10 = memory[x5 + 0] = memory[10] = 15
sd x6, 24(x5)            # x6 is stored at memory address x5 + 24 (10 + 24 = 34)
ld x11, 24(x5)           # x11 = memory[x5 + 24] = memory[34] = -5
beq x4, x5, 8           # if x4 == x5, branch to PC + 8 (so skip the next instruction)
beq x0, x0, 0           # if x0 == x0, branch to PC + 0 (infinite loop)
add x13, x1, x10        # x13 = x1 + x10 = 30
```

Figure 16: Assembly Instructions for Test_Basic_Code

Register File Output

```
==== REGISTER FILE CONTENTS ====
x1  = 000000000000000f
x2  = fffffffffffffffb
x3  = 000000000000000a
x4  = 000000000000000a
x5  = 000000000000000a
x6  = fffffffffffffffb
x7  = fffffffffffffffb
x10 = 000000000000000f
x11 = fffffffffffffffb
x13 = 000000000000001e

All other registers = 0000000000000000

Total instructions executed: 15
```

4 Testing and Results

To verify the correct functionality of the sequential RISC-V processor, a Fibonacci program was executed. The program computes the 10th Fibonacci number using a loop and branch instructions.

Test Program (Fibonacci Sequence)

Listing 25: Assembly Program for Fibonacci Computation

```
addi x1, x0, 10        # Initialize loop counter n = 10
addi x2, x0, 0         # a = 0
addi x3, x0, 1         # b = 1

addi x1, x1, -1        # n = n - 1
beq  x1, x0, 20        # If n == 0, exit loop

add  x4, x2, x3        # temp = a + b
addi x2, x3, 0         # a = b
addi x3, x4, 0         # b = temp

beq  x0, x0, -20       # Unconditional branch back to loop
```

Register File Output

```
==== REGISTER FILE CONTENTS ====
x0  = 0000000000000000
x1  = 0000000000000000
x2  = 0000000000000022
x3  = 0000000000000037
x4  = 0000000000000037
All other registers = 0000000000000000

Total instructions executed: 60
```

Explanation

The program initializes registers to generate the Fibonacci sequence. Register x1 acts as a loop counter and is decremented in each iteration. The branch instruction `beq x1, x0, 20` terminates the loop when the counter reaches zero.

Registers x2 and x3 store consecutive Fibonacci numbers, while x4 temporarily stores their sum. The unconditional branch `beq x0, x0, -20` ensures the loop repeats until completion.

After 10 iterations:

- x2 = 34 (9th Fibonacci number)
- x3 = 55 (10th Fibonacci number)

- x4 = 55 (last computed value)
- x1 = 0 (loop counter exhausted)

The total instruction count of 60 confirms correct loop execution (6 instructions per iteration $\times$ 10 iterations).

5 Conclusion

In this project, a sequential (non-pipelined) RISC-V processor based on the RV64I instruction set was successfully designed and implemented using Verilog. The processor executes instructions in a single-cycle flow, completing instruction fetch, decode, execute, memory access, and write-back stages sequentially for each instruction.

All major components of the datapath, including the Program Counter, Instruction Memory, Control Unit, Register File, Immediate Generator, ALU Control, Arithmetic Logic Unit, Data Memory, multiplexers, and adder units, were developed as independent modules and then integrated to form a complete processor. The design supports arithmetic, logical, memory, and branch instructions such as add, sub, addi, and, or, ld, sd, and beq.

Simulation results verified correct functionality of each module as well as the integrated system. The processor correctly fetched and executed instructions, performed memory operations, handled branch conditions, and updated register values as expected.

This project provided a comprehensive understanding of processor architecture, datapath design, control logic, and hardware description using Verilog. It also demonstrated the importance of modular design and systematic verification in building complex digital systems.